

Quickeee - Gen AI and Data Engineer Assignment

The Challenge: The Luxury Stylist Concierge

The Scenario

Quickeee is launching an ultra-premium fashion concierge. Users don't just buy clothes; they ask our AI to style them. We need a backend system that can dynamically scrape real-time inventory from premium clothing brands, store it intelligently, and use an agentic LLM workflow to recommend the perfect outfit.

You need to build a context-aware shopping assistant pipeline that can take a user's prompt (e.g., "I have dark navy chinos, what t-shirt and shoes should I wear for a summer yacht party?"), retrieve matching inventory, and return a flawlessly reasoned fashion pairing via API.

Technical Requirements

1. Advanced Scraping & Data Pipeline (The Foundation)

- Target at least two public fashion/apparel websites (e.g., Zara, Myntra, H&M, or any luxury equivalent).
- Build a robust scraper (using Playwright, Scrapy, or equivalent) to extract at least 50 tops (t-shirts, shirts) and 50 bottoms (pants, shorts).
- Extract clean JSON containing the Item Name, Price, Image URL, Category, and Description.
- The scraper must be designed to bypass basic rate limits and handle complex DOM structures gracefully.

2. RAG & Vector Databases (The Memory)

- Generate embeddings for your scraped catalog and push them to a Vector Database (Pinecone, Milvus, Qdrant, or local ChromaDB).
- Structure the metadata so the AI can filter by price, category, or color before doing a semantic search.

3. The AI Concierge & Token Economics (The Brain)

- Deploy an Agentic workflow (LangChain, LangGraph, or native OpenAI/Gemini routing).
- When a user asks for an outfit recommendation, the system must use RAG to query the Vector DB for available, relevant items.

- The LLM must evaluate fashion rules (e.g., matching the right t-shirt with the requested pants) and return a final JSON payload containing the recommended items, total price, and a short, luxurious "Stylist Note" explaining why the pieces work together.
- Demonstrate a Frugal Mindset: Implement a semantic cache or show prompt optimization techniques to prove you treat token costs like your own money.

4. Seamless Integration (The Delivery)

- Expose this entire workflow via a single FastAPI endpoint (POST /api/v1/style-me).
- The endpoint must accept a user prompt and return the structured JSON payload.

Submission Guidelines

- **Repository:** Provide the source code via a public GitHub repository.
- **Documentation:** Crucial: Include a brief ARCHITECTURE file explaining your state management choices, database schema, prompt optimization strategies, and how you structured your folders.
- **System Flow:** Include a flowchart (image or Mermaid.js) mapping out the complete data journey from the user's prompt, through the RAG pipeline, to the final API response.
- **Demo:** Include a screen recording of the app running, showing a start-to-finish demo. Show the FastAPI Swagger UI or Postman, send a complex prompt, and show the resulting JSON output and terminal logs detailing the agent's thought process.